Ten Rules for Governed Coding (A Companion to the Power of Ten)

Minimal Invariants for Epistemic Safety in Agentic Systems

Safety-critical software engineering has long recognized that expressiveness must sometimes be constrained to achieve analyzability.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Safety-critical software engineering has long recognized that expressiveness must sometimes be constrained to achieve analyzability. Gerard J. Holzmann's "Power of Ten" rules formalized this discipline at the level of runtime behavior: bounded control flow, restricted memory use, and zero-warning static analysis. These constraints reduce catastrophic execution failure.

Modern agentic systems introduce a distinct class of risk. They generate configuration, invoke heterogeneous tools, reinterpret policy, and accumulate side effects under shifting authority. Failures arise not only from unsafe execution but from implicit legitimacy, nondeterministic mutation, silent escalation, and unrecorded effects. Runtime safety does not guarantee epistemic safety.

This paper proposes a companion discipline: **Ten Rules for Governed Coding**, a minimal, mechanically enforceable invariant set for systems in which meaning, authority, and side effects must remain analyzable. The goal is not to replace runtime safety constraints, but to extend safety-critical thinking into the epistemic layer. If Holzmann constrains how code runs, these rules constrain how meaning is allowed to act.

---

1. The Missing Layer

Holzmann's rules assume stable, legitimate program meaning and constrain execution to ensure predictability and analyzability.

In agentic and policy-bearing systems, that assumption no longer holds.

Modern systems:

- dynamically generate code and configuration

- compose tools across trust boundaries
- modify policy artifacts
- interpret prompts as executable instructions
- interact continuously with external substrates

The resulting failures are epistemic:

- authority drift
- implicit policy mutation
- nondeterministic state transitions
- unreceipted side effects
- retroactive reinterpretation of history

The question is no longer only:

> Will the program terminate safely?

It is also:

> Was this action legitimate under declared constraints? Can the system demonstrate-mechanically-why it changed?

The following rules constrain this epistemic layer.

---

2. Ten Rules for Governed Coding

The rules are intentionally few, strict, and designed for mechanical enforcement.

---

1. Canonical Meaning Only

Rule: All authoritative meaning must be representable in a single canonical semantic intermediate representation (IR). No behavior-defining content may exist solely in prompts, ad hoc configuration, undocumented conventions, or backend interpretation. *Rationale:* If meaning is not canonically representable, it cannot be diffed, hashed, replayed, or verified. Hidden semantics are indistinguishable from hidden authority.

---

2. No Action Without Admission

Rule: No artifact may act without explicit admission under a declared law artifact. Admission must bind to immutable identifiers (e.g., law version or hash) and produce a recordable verdict. *Rationale:* Execution without admission is implicit authority. Binding admission to a specific law version prevents contextual or retroactive legitimacy.

---

3. Bounded Episodes Only

Rule: Canonical state may change only through bounded episodes. Each episode must declare its inputs, intended deltas, and termination conditions. Commits outside episode scope are forbidden. *Rationale:* Unscoped mutation destroys lineage. Episodes convert change into an analyzable unit tied to explicit cause.

---

4. Replayability Is Mandatory

Rule: Every episode must be replayable from declared inputs. All nondeterminism (time, randomness, concurrency races, external responses) must be eliminated or captured and bound to the episode record. *Rationale:* If behavior cannot be reproduced, it cannot be proven. Replayability transforms execution into evidence.

---

5. Effects Through a Membrane

Rule: All external effects must pass through an explicit, declared effect membrane. Undeclared side-effect channels are forbidden. *Rationale:* Side effects are the primary vehicle of implicit authority. A membrane renders effects inspectable and bounded.

---

6. Explicit, Non-Transitive Authority

Rule: Authority must be explicit, minimal, and non-transitive. No operation may rely on privileges implicitly inherited from environment, role, or context. Authority tokens must be auditable and scope-bound. *Rationale:* Silent privilege escalation is the epistemic analogue of memory corruption. Authority must be consumed deliberately.

---

7. No Retroactive Policy Drift

Rule: Historical admissibility decisions remain bound to the law artifact used at the time of admission. Policy updates may not reinterpret past artifacts without creating a new episode and admission record. *Rationale:* Retroactive reinterpretation collapses lineage into revisionism. Governance requires immutable historical binding.

---

8. All Outcomes Must Be Handled

Rule: All non-void outcomes-return values, validation results, effect receipts-must be checked, propagated, or explicitly marked as intentionally ignored in a machine-detectable manner. *Rationale:* Ignored outcomes conceal failure. Explicit handling converts omission into a conscious, auditable decision.

---

9. Typed, Hash-Stable Canonicalization

Rule: Canonical artifacts must be typed, minimal, and hash-stable. Meaning must not depend on formatting, presentation structure, or transport encoding. Canonicalization must be deterministic. *Rationale:* If identical meaning can serialize differently, identity and integrity become ambiguous. Hash-stability anchors lineage.

---

10. Zero-Ambiguity Standard

Rule: The system must fail closed on unresolved warnings from canonicalizers, analyzers, admissibility checks, or effect audits. If tooling cannot confidently validate an invariant, the artifact must be rewritten until it becomes trivially analyzable. *Rationale:* Warnings represent uncertainty. In governed systems, uncertainty is a defect. Artifacts must be easy to verify, not merely plausible.

---

3. Mechanical Enforcement

Each rule implies concrete enforcement mechanisms:

- canonical IR completeness validation
- admission gate enforcement
- episode boundary enforcement
- deterministic replay tests
- effect capability audits
- authority token flow analysis
- law/version binding verification
- outcome-handling linting
- deterministic canonicalization and hashing tests
- zero-warning governance pipelines

These checks are structural rather than aspirational.

---

4. Rule Summary

- **Rule:** 1. Canonical Meaning Only
  Primary Check: IR completeness validation
  Failure Class Prevented: Hidden semantics
- **Rule:** 2. No Action Without Admission
  Primary Check: Admission gate enforcement
  Failure Class Prevented: Implicit authority
- **Rule:** 3. Bounded Episodes Only
  Primary Check: Episode boundary enforcement
  Failure Class Prevented: Untracked mutation
- **Rule:** 4. Replayability Is Mandatory
  Primary Check: Deterministic replay tests
  Failure Class Prevented: Unverifiable history
- **Rule:** 5. Effects Through a Membrane
  Primary Check: Effect capability audits
  Failure Class Prevented: Ambient side effects
- **Rule:** 6. Explicit Authority
  Primary Check: Authority flow analysis
  Failure Class Prevented: Silent escalation
- **Rule:** 7. No Retroactive Policy Drift
  Primary Check: Law/version binding validation
  Failure Class Prevented: Historical reinterpretation
- **Rule:** 8. All Outcomes Must Be Handled
  Primary Check: Outcome-handling linting
  Failure Class Prevented: Concealed failure
- **Rule:** 9. Typed, Hash-Stable Canonicalization
  Primary Check: Deterministic canonicalization tests
  Failure Class Prevented: Identity ambiguity
- **Rule:** 10. Zero-Ambiguity Standard
  Primary Check: Zero-warning governance pipeline
  Failure Class Prevented: Analytical uncertainty

---

5. Relationship to Runtime Safety

These rules do not replace runtime safety disciplines such as bounded control flow, restricted memory usage, or static analysis constraints. They operate at a distinct layer.

Runtime safety ensures that systems behave predictably during execution.

Epistemic safety ensures that systems change only under demonstrable authority, produce replayable evidence, and preserve lineage integrity.

Runtime safety without epistemic safety yields reliable drift. Epistemic safety without runtime safety yields legitimate crashes.

The disciplines are complementary.

---

6. Conclusion

Modern computational systems no longer merely execute instructions; they interpret, compose, authorize, and persist decisions across boundaries of trust. Safety must therefore extend beyond control flow into governance.

These ten rules restrict expressiveness to preserve analyzability of authority, meaning, and effects. They trade convenience for replayability, implicit trust for explicit admission, and ambient capability for bounded interaction.

When systems act autonomously and persist beyond direct human oversight, the decisive question is not only whether they run safely, but whether they can demonstrate-mechanically-why they were allowed to run at all.

Corpus-Wide Reference Positioning

This paper argues that safety-critical software engineering has long recognized that expressiveness must sometimes be constrained to achieve analyzability. Its nearest external comparison fields are cps, software, security. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Runtime monitors and fallback controllers only protect modeled safety envelopes and can fail through sensor error, timing assumptions, unsafe recovery, or omitted hazards. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Edward R. Griffor et al. *Framework for Cyber-Physical Systems: Volume 1, Overview*. NIST SP 1500-201, 2017. DOI: 10.6028/NIST.SP.1500-201. https://doi.org/10.6028/NIST.SP.1500-201 **Relationship:** Adjacent cyber-physical systems framework for trustworthiness, timing, data, and lifecycle concerns.

[**E2**] J. Tanner Slagel et al. "A Formal Verification Framework for Runtime Assurance." *NASA Formal Methods*, 2024. NASA NTRS 20230017350. https://ntrs.nasa.gov/citations/20230017350 **Relationship:** Adjacent runtime-assurance formalization, useful for comparing failover safety with constitutional admission and effect recognition.

[**E3**] Nancy G. Leveson and John P. Thomas. *STPA Handbook*. MIT, 2018. https://psas.scripts.mit.edu/home/books-and-handbooks/ **Relationship:** Adjacent systems-theoretic hazard analysis method; used to test treatment of unsafe control and system-level constraints.

[**E4**] Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Peter G. Neumann. *An Introduction to CHERI*. University of Cambridge Technical Report UCAM-CL-TR-941, 2019. DOI: 10.48456/tr-941. cl.cam.ac.uk source **Relationship:** Adjacent capability architecture demonstrating unforgeable hardware authority and compartmentalization.

[**E5**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E6**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E7**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

Ableman corpus

[**A1**] Adam Ableman. *Constitutional Robotics.* Ableman Corpus current edition; work identity `PAPER.PUB.18750683`;
source SHA-256
`D10DF9B6B7F742D9EEABF01BC5DFC3755BB4EBF548E8CB5F215BB9D94678F514`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *A Constitutional Syscall Architecture for Cyber-Physical Control Systems.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-F46D86D479F89BAC`;
source SHA-256
`5E696B753A10BD3C3D36B675A84B95CCF2234F024A6B277BF121E76BFCBCBD18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Effect Sovereignty.* Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Constitutionally Governed Infrastructure.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-EF5747107D56F516`;
source SHA-256
`9C708BA381108EEA252404DF33F7B3B1648EC9CF5F2ACF00BAB88C6A15540926`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Constitutional Syscalls.* Ableman Corpus current edition; work identity `PAPER.PUB.18737229`;
source SHA-256
`5F14356180812262778F8574C292F4F6B560993042E5358B02EAB9CF0F7DE14C`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;

source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18624994